

Topic 35 — Dockerized Environment Setup for LLMs

Docker from absolute zero — no prior container knowledge assumed.



New to Docker



Intermediate



Expert



Interview Q&A



Layer 1 — New to Docker (from absolute zero)

Part 1 — What problem is this even solving?

Imagine you get Ollama + FastAPI working perfectly on your laptop. You send the project to a teammate. On their machine:

- They have Python 3.9, you have 3.12 — half your code won't run
- A package needs a C++ compiler they don't have
- Ollama is on port 11434 for you; they already have something else there
- They're on Windows, you're on Mac — file paths differ

You spend two days on a setup call instead of building. This is the **"works on my machine"** problem, and it gets worse with every service you add. An LLM stack has three services minimum.

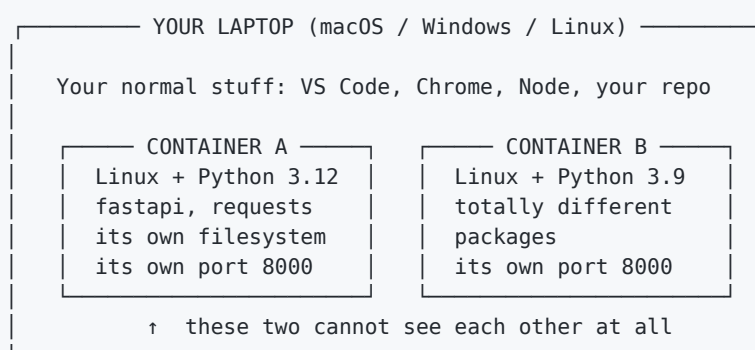


Docker's answer

Stop shipping *instructions* for setting up an environment. Ship the **entire environment itself** — operating system files, Python, packages, config, everything — as a single portable bundle. Your teammate runs one command and gets a byte-for-byte identical setup.

Part 2 — What a container actually is

A **container** is a sealed box running on your computer, with its own miniature Linux system inside. It has its own filesystem, its own installed programs, its own network address, its own view of the world. Programs inside it genuinely believe they're alone on a fresh machine.



Two containers can each run a different Python version, with conflicting packages, both listening on "port 8000" internally, and neither knows the other exists. Nothing they install touches your real machine.

✔ "So it's a virtual machine?"

Close, but importantly lighter. A VM boots a whole second operating system: gigabytes of disk, a minute to start, its own kernel. A container **shares your machine's kernel** and only packages the layer above it. Result: megabytes instead of gigabytes, starting in under a second. That difference is why running five containers at once is normal and running five VMs is not.

Part 3 — Image and container: the class/object mapping

This is the single most important distinction in Docker, and you already know it from **Topic 14**.

```
class PythonApp:                                # ← the blueprint. Defined once.
    def __init__(self):
        self.python = "3.12"
        self.packages = ["fastapi", "requests"]

app1 = PythonApp()                               # ← a live instance
app2 = PythonApp()                               # ← another, independent instance
```

```
docker build -t python-app .                     # ← create the blueprint = IMAGE
docker run python-app                             # ← instantiate it = CONTAINER
docker run python-app                             # ← instantiate again = another CONTAINER
```

Docker	OOP	Why it maps
Image	Class	A blueprint. Immutable, sits on disk, does nothing until instantiated.
Container	Object	A live instance with its own memory, filesystem, and running process.
<code>docker run</code>	<code>PythonApp()</code>	The instantiation call.
Dockerfile	The class definition	The source text that produces the blueprint.
<code>-e VAR=value</code>	Constructor arguments	Per-instance config, like <code>PythonApp(env="prod")</code> .
Volume	An injected database	External state that outlives the instance.

✂ Every OOP relationship carries over

One class → many objects. One image → many containers, mutually isolated.

Deleting an object doesn't delete the class. `docker rm` removes a container; the image survives, ready to spawn more.

Each object has its own state. Write a file inside container A — container B, from the same image, never sees it.

Modifying an instance doesn't change the blueprint. `pip install` inside a running container is gone at the next `docker run`, exactly as setting an attribute on one object leaves the class untouched.

Where does the image come from?

- Download one someone else built** — from Docker Hub, a public registry. `ollama/ollama` is a ready-made image the Ollama team publishes. Like `npm install`, but for entire environments.
- Build your own** from a Dockerfile — which is what you'll do for your FastAPI service.

Part 4 — The Dockerfile: writing the class definition

A **Dockerfile** is a plain text file, literally named `Dockerfile`, no extension. It's a recipe read top to bottom.

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Instruction	What it means
<code>FROM python:3.12-slim</code>	"Start from a pre-made image that already has Linux + Python 3.12." Every Dockerfile starts by inheriting from something.
<code>WORKDIR /app</code>	"Create <code>/app</code> inside the container and treat it as the current directory." Like <code>cd</code> , for every command after it.
<code>COPY requirements.txt .</code>	"Copy this file from my laptop into the container." One-directional, at build time.
<code>RUN pip install ...</code>	"Execute this while building the image ." The result is baked in permanently.
<code>COPY . .</code>	"Now copy all my source code in."
<code>CMD ["uvicorn", ...]</code>	"When someone instantiates this image, run this — the container's one job."

The `FROM` line is the OOP parallel again, and it isn't loose — it's genuinely inheritance:

```
FROM python:3.12-slim # inherit
RUN pip install fastapi # extend
```

```
class MyApp(PythonBase): # identical
    ... # shape
```

⚠ RUN vs CMD — the distinction that confuses everyone

RUN executes at **build time**, once, and its effect is frozen into the image. Think of it as code in the class *body*.

CMD executes at **run time**, every time a container starts. Think of it as the method that runs on instantiation.

When the CMD process exits, **the container stops**. A container isn't a machine that stays up — it's a wrapper around one process. Process ends, container ends.

Part 5 — The commands you'll actually type

```
docker build -t my-api . # read the Dockerfile here, name the image "my-api"
docker run my-api # start a container from it
docker ps # what's running right now
docker ps -a # including stopped ones
docker images # blueprints on disk
docker stop my-api # stop a running container
docker rm my-api # delete the container (image untouched)
docker logs -f my-api # tail its output
docker exec -it my-api bash # open a shell INSIDE the container
```

Your main debugging tool

`docker exec -it <name> bash` drops you into a terminal *inside the sealed box* — you can `ls`, `cat` files, `curl` other services, and see exactly what the container sees. When something mysteriously doesn't work, go look from the inside.

Compare the vocabulary to Topic 34

Ollama	Docker
<code>ollama pull</code>	<code>docker pull</code>
<code>ollama run</code>	<code>docker run</code>
<code>ollama list</code>	<code>docker images</code>
<code>ollama ps</code>	<code>docker ps</code>
<code>ollama rm</code>	<code>docker rm</code>
<code>Modelfile</code>	<code>Dockerfile</code>
<code>ollama create</code>	<code>docker build</code>

Ollama deliberately copied this grammar. Learning one taught you most of the other.

Part 6 — Ports: how anything reaches inside the box

The box is sealed. By default, a program listening inside a container is **completely unreachable from your laptop**. You have to punch a hole.

```
docker run -p 8000:8000 my-api
```

```
      -p 8000:8000
        ↑   ↑
      HOST  CONTAINER
      (your  (inside
      laptop) the box)
```

"Traffic arriving at port 8000 on my laptop → forward it to port 8000 inside this container."

The two numbers are independent. `-p 3000:8000` means you visit `localhost:3000` in your browser while the container still believes it's serving on 8000. Handy when two containers both insist on port 8080 — give them different host ports.

Without `-p`, your API runs perfectly and you cannot reach it

No error message, just a refused connection. This is the first thing to check when a container "doesn't work."

Part 7 — Volumes: why your data keeps vanishing

Container filesystems are **temporary**. Delete the container and everything written inside it is gone. That's usually the point — clean slate every time. But sometimes you need data to survive.

The OOP version of the problem

```
class ModelStore:
    def __init__(self):
        self.weights = {}          #  dies with the object
```

```
class ModelStore:
    def __init__(self, db):
        self.db = db  #  external store – outlives the object
```

A volume is that external store.

```
docker run -v ollama_data:/root/.ollama ollama/ollama
```

#

storage name *path inside container*

"Anything written to `/root/.ollama` inside the container — put it in a persistent store named `ollama_data` instead."

 The mistake that costs 5 GB

Ollama saves model weights to `/root/.ollama`. Without the volume, deleting the container means **re-downloading every model**. With it, the next container picks them straight up.

 **The mistake that costs 5 GB**

Ollama saves model weights to `/root/.ollama`. Without the volume, deleting the container means **re-downloading every model**. With it, the next container picks them straight up.

Two flavours

```
-v ollama_data:/root/.ollama      # named volume – Docker manages the storage
-v ./api:/app                     # bind mount – maps a real folder on your disk
```

	Named volume	Bind mount
Where it lives	Docker's internal storage	An actual path on your disk
Speed	Fast everywhere	Slow on macOS / Windows
Edit it directly?	Awkward	Yes — that's the point
Use for	Model weights, database files	Source code you're editing

✅ **The rule**

Named volume for model weights. Bind mount for code you're editing. The bind mount is what gives you **hot reload** — your `./api` folder *is* the container's `/app`, so saving in VS Code is instantly visible inside and `uvicorn --reload` restarts.

✅ **The rule**

Named volume for model weights. Bind mount for code you're editing. The bind mount is what gives you **hot reload** — your `./api` folder *is* the container's `/app`, so saving in VS Code is instantly visible inside and `uvicorn --reload` restarts.

Part 8 — The full command, now readable

```
docker run -d \
  --name ollama \
  -p 11434:11434 \
  -v ollama_data:/root/.ollama \
  -e OLLAMA_HOST=0.0.0.0 \
  -e OLLAMA_KEEP_ALIVE=30m \
  ollama/ollama
```

Flag	Plain English	If you skip it
<code>-d</code>	Run in background	Your terminal is stuck
<code>--name ollama</code>	Call it "ollama"	Docker names it <code>sad_hopper</code>
<code>-p 11434:11434</code>	Open a hole to port 11434	Nothing can reach it

Flag	Plain English	If you skip it
<code>-v ollama_data:...</code>	Persist the model weights	Re-download 5 GB every restart
<code>-e OLLAMA_HOST=0.0.0.0</code>	Set an env var inside	Nothing can reach it (Part 9)
<code>ollama/ollama</code>	Which image to instantiate	—

`-e` sets environment variables inside the container — the **constructor arguments** of the analogy. Same idea as `process.env` in Node or `os.environ` in Python.

Part 9 — `0.0.0.0` : the failure that will cost you an hour

You've mapped the port correctly and it still refuses connections. Here's why.

When a program starts a server, it chooses **which network interface to listen on**:

- `127.0.0.1` (localhost) — "only accept connections from this same machine"
- `0.0.0.0` — "accept connections arriving on any interface"

On your laptop, `127.0.0.1` is fine — you *are* the same machine. **Inside a container, "this same machine" means the container**, not your laptop. Your port mapping delivers traffic to the container's external interface, the program isn't listening there, connection refused.

- ✗ binds `127.0.0.1` → listening only on the container's internal loopback
→ your `-p` mapping delivers to a door nobody is behind
- ✓ binds `0.0.0.0` → listening on all interfaces, including the mapped one

This applies to **everything you containerize**:

```
uvicorn main:app --host 0.0.0.0 --port 8000 # ✓ in Docker
uvicorn main:app                          # ✗ defaults to 127.0.0.1
```

💡 You may have hit this already

If you've ever deployed a Next.js app to a container and gotten a blank page, this was almost certainly it.

Part 10 — Docker Compose: the point of the whole topic

Your Phase 5 stack has three services:

```

Ollama (the model runtime) ↔ OpenWebUI (browser chat)
                             ↑
                        FastAPI (your Python API)
```

Three `docker run` commands, each with six flags, in the right order, every time you sit down to work. Nobody does that.

Docker Compose puts it all in one file. Roughly: `docker run` is `node script.js`; Compose is `package.json` scripts plus a dependency graph.

```
# docker-compose.yml
services:

  ollama:
    image: ollama/ollama           # use the pre-built image
    container_name: ollama
    ports:
```

```

- "11434:11434"          # = -p
volumes:
  - ollama_data:/root/.ollama    # = -v
environment:
  - OLLAMA_HOST=0.0.0.0          # = -e
  - OLLAMA_KEEP_ALIVE=30m
restart: unless-stopped          # auto-restart if it crashes

openwebui:
  image: ghcr.io/open-webui/open-webui:main
  container_name: openwebui
  ports:
    - "3000:8080"                # browser hits 3000, app serves 8080
  volumes:
    - openwebui_data:/app/backend/data
  environment:
    - OLLAMA_BASE_URL=http://ollama:11434
  depends_on:
    - ollama                      # start ollama first
  restart: unless-stopped

volumes:                          # declare the named volumes
  ollama_data:
  openwebui_data:

```

Every line maps to a flag you already met. Then:

```

docker compose up -d          # start everything
docker compose ps             # status
docker compose logs -f        # tail all logs together
docker compose down           # stop + remove containers (volumes survive)
docker compose down -v        # ⚠️ ALSO deletes volumes — you lose the models

```

💣 **docker compose down -v is destructive**

-v is muscle memory from other tools, and here it deletes your volumes — meaning you re-download gigabytes of model weights. Think before you type it.

Part 11 — How containers find each other

This is the second big gotcha, and it follows directly from the sealed-box idea. Look at the line that makes OpenWebUI work:

```
OLLAMA_BASE_URL=http://ollama:11434
```

Not `localhost`. `ollama` — the service name from the YAML.

Compose builds a private network for your services and registers **each service name as a hostname** on it. Inside that network, `http://ollama:11434` resolves to the Ollama container automatically.

⚠️ Two details that matter

- 1. localhost inside a container means that container itself.** If OpenWebUI called `http://localhost:11434`, it would search inside its own box, find nothing, and fail.
- 2. Use the container port, not the host mapping.** Container-to-container traffic never touches your host's ports. Even if you'd mapped `-p 9999:11434`, other containers still use 11434.

Your browser → localhost:3000 → [openwebui container]

http://ollama:11434
(private Compose network)
▼
[ollama container]

Calling from	Use
Your laptop / browser	<code>http://localhost:11434</code>
One container → another in the same Compose file	<code>http://ollama:11434</code>
A container → something running on your host	<code>http://host.docker.internal:PORT</code>

Part 12 — Adding your own service

For your FastAPI app there's no pre-built image, so `build:` replaces `image:` — "construct this blueprint from a Dockerfile in that folder."

```
api:
  build: ./api                # ← build, don't download
  container_name: api
  ports:
    - "8000:8000"
  volumes:
    - ./api:/app              # bind mount → hot reload
  environment:
    - OLLAMA_BASE_URL=http://ollama:11434
  depends_on:
    - ollama
```

With `api/Dockerfile` being the file from Part 4. Your project ends up:

```
project/
├── docker-compose.yml
│   └── api/
│       ├── Dockerfile
│       ├── requirements.txt
│       └── main.py
```

And your Python reads the URL from the environment rather than hardcoding it — the same pattern that lets you swap to OpenAI later:

```
import os

OLLAMA_URL = os.getenv("OLLAMA_BASE_URL", "http://localhost:11434")
```

The default keeps it working when you run outside Docker; Compose overrides it inside.

Part 13 — Installing Docker

Platform	Install	Note
Windows	Docker Desktop	Enable the WSL 2 backend in Settings → General. The old Hyper-V backend is slower.
macOS	Docker Desktop	Pick the Apple Silicon build if you're on M-series.
Linux	Docker Engine + <code>docker-compose-plugin</code>	No Desktop needed.


```
docker --version
docker compose version    # "compose", not "docker-compose" – v2 syntax
docker run hello-world    # verify end to end
```

✅ What hello-world proves

If it prints a greeting, Docker downloaded an image, instantiated a container, ran its CMD, and exited. That's the entire lifecycle in one command.

Layer caching — why dependency-copy comes first

A Docker image is a **stack of read-only layers**, one per instruction. Docker caches each layer and reuses it if that instruction *and everything above it* are unchanged.

```
# ❌ Slow — any code edit reinstalls every dependency
COPY . .
RUN pip install -r requirements.txt

# ✅ Fast — deps only reinstall when requirements.txt changes
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
```

With the wrong order, changing one line of Python invalidates the `COPY . .` layer and every layer after it — a 90-second rebuild. With the right order, it's two seconds.

💡 Same reasoning you've seen in Node

`COPY package*.json ./` before `COPY . .` exists for exactly this reason. Dependency manifest first, source second — universal Dockerfile grammar.

`.dockerignore`

Does for build context what `.gitignore` does for commits:

```
.venv/
__pycache__/
*.pyc
.git/
.env
```

Without it, Docker ships your entire virtualenv into the build context on every build — slow, and it can leak secrets into the image.

GPU passthrough

CPU inference works but is slow. On an NVIDIA machine:

```
ollama:
  image: ollama/ollama
  deploy:
    resources:
      reservations:
        devices:
          - driver: nvidia
            count: all
            capabilities: [gpu]
```

This needs the **NVIDIA Container Toolkit** on the host — the container gets a device handle, but the driver stays on the host. Verify with:

```
docker exec ollama nvidia-smi
```

Docker on macOS runs inside a Linux VM that has **no access to the Metal GPU**. Ollama in Docker on a Mac is CPU-only and noticeably slower.

The pragmatic macOS setup: Ollama installed **natively** on the host, everything else in Docker, with containers reaching it via `http://host.docker.internal:11434`.

Resource limits and the OOM kill

LLM containers will happily consume all your RAM and get killed by the kernel — often silently, showing up as a container that mysteriously restarted.

```
ollama:
  deploy:
    resources:
      limits:
        memory: 12G
      reservations:
        memory: 8G
```

⚠ Exit code 137

That's SIGKILL from the kernel's OOM killer. Diagnose with:

```
docker inspect ollama --format '{{.State.OOMKilled}}'
```

Health checks and real startup ordering

`depends_on` alone only waits for the container to *start*, not to become *ready*. Your FastAPI service will fire requests at an Ollama that's still loading weights.

```
ollama:
  healthcheck:
    test: ["CMD-SHELL", "ollama list || exit 1"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 30s

api:
  depends_on:
    ollama:
      condition: service_healthy  # ← now it actually waits
```

✅ Still build retry logic into your client

Health checks reduce races; they don't eliminate them. A model cold-load (Topic 34) can exceed any reasonable healthcheck timeout, so your Python should retry on connection errors regardless.

Where the weights actually live

```
docker volume ls
docker volume inspect ollama_data
du -sh $(docker volume inspect ollama_data -f '{{.Mountpoint}}')
```

Models are gigabytes each. Two practical consequences:

- `docker compose down -v` is a **destructive command** — it deletes the volume and you re-download everything. Muscle-memory `-v` from other projects will cost you an hour.
- To keep models on an external drive, use a bind mount instead: `-v /Volumes/ext/ollama:/root/.ollama`

The security posture

Concern	Reality
Port exposure	<code>-p 11434:11434</code> on a server binds <code>0.0.0.0</code> on the <i>host</i> too — and Ollama has no authentication whatsoever . Use <code>-p 127.0.0.1:11434:11434</code> to restrict to localhost.
Root by default	Containers run as root. Add a non-root <code>USER</code> in production Dockerfiles.
Secrets	Never <code>ENV OPENAI_API_KEY=sk-...</code> in a Dockerfile — it's baked into an image layer and readable via <code>docker history</code> . Use <code>env_file</code> or Docker secrets.

The one that ends up in incident reports

An internal LLM endpoint exposed on a cloud VM's public interface with no authentication in front of it. Port 11434 open to the world means anyone can use your compute, read your loaded models, and pull new ones. **Bind to 127.0.0.1 on any shared machine.**

Compose profiles — one file, several setups

```
openwebui:
  profiles: ["ui"]
```

```
docker compose up -d          # just ollama + api
docker compose --profile ui up -d  # + OpenWebUI
```

Useful when your CI needs the API and Ollama but not a browser UI.

Interview Q&A

Q: Explain the difference between a Docker image and a container.

An image is an immutable blueprint stored on disk; a container is a running instance of it. The cleanest framing is class and object — one image can produce many containers, each with its own filesystem, memory, and process, all isolated from each other. Deleting a container leaves the image intact, and changes made inside a running container don't propagate back to the image, exactly as setting an attribute on an object doesn't modify its class.

Q: Why containerize an LLM stack rather than installing everything natively?

Reproducibility and isolation. An LLM stack combines Python, system libraries, GPU drivers, and multiple services that must discover each other — Docker pins all of that into a versioned artifact that behaves identically on a laptop, in CI, and on a server. It also isolates conflicting dependencies, and Compose turns multi-service networking into declarative config instead of manual setup steps in a README.

Q: Your OpenWebUI container can't reach Ollama, though both are running. Where do you look?

Two things, in order. First, whether OpenWebUI is pointed at `localhost` instead of the service name — `localhost` inside a container means that container itself, so it must be `http://ollama:11434`, using Compose's internal DNS and the container port. Second, whether Ollama is bound to `0.0.0.0` rather than `127.0.0.1`; loopback binding inside a container rejects everything arriving through the port mapping. I'd confirm with `docker exec openwebui curl http://ollama:11434/api/tags`.

Q: Named volume or bind mount for model weights — and why?

A named volume is Docker-managed storage referenced by name; a bind mount maps a specific host path into the container. Model weights go in a **named volume** — better performance, especially on macOS and Windows where bind mounts cross a VM boundary, and no dependence on host paths. Bind mounts are for source code you're actively editing, since edits appear in the container immediately and enable hot reload.

Q: Why does `COPY requirements.txt` come before `COPY . .` in a Dockerfile?

Layer caching. Docker invalidates a layer and everything after it when its inputs change. Copying the whole source before installing means every code edit reinstalls all dependencies. Copying only the dependency manifest first means the expensive install layer is cached and reused until the manifest itself changes — the same reason Node Dockerfiles copy `package.json` before the source.

Q: You're on an M3 MacBook. Should you run Ollama in Docker?

Probably not. Docker on macOS runs in a Linux VM with no access to Apple's Metal GPU, so containerized Ollama is CPU-only and substantially slower. The better setup is Ollama installed natively on the host, with the rest of the stack in Docker reaching it via `host.docker.internal:11434`. On a Linux machine with an NVIDIA GPU, containerizing Ollama is fine — you pass the GPU through with the NVIDIA Container Toolkit.

MEMORY HOOK

Image = class. Container = object.

Ports for humans, service names for containers.

`localhost:3000` from your browser · `http://ollama:11434` from another container
...and `0.0.0.0`, or nothing listens at all.